第29章源码阅读

由正式源文件生成的静态阅读副本。返回正文 · 运行说明

---

src/main/java/org/example/jvminternals/ch29/ContextJourney.java

```
package org.example.jvminternals.ch29;

/** 普通编译展示线程局部清理与正式作用域值，不依赖预览接口。 */
public final class ContextJourney {
    private static final ThreadLocal<String> LOCAL = new ThreadLocal<>();
    private static final ScopedValue<String> REQUEST = ScopedValue.newInstance();

    private ContextJourney() { }

    /** 嵌套绑定恢复，线程局部值由拥有者显式移除。 */
    public static void main(String[] args) {
        LOCAL.set("legacy-context");
        try {
            System.out.println("线程局部: " + LOCAL.get());
            ScopedValue.where(REQUEST, "outer").run(() -> {
                show("外层");
                ScopedValue.where(REQUEST, "inner").run(() -> show("内层"));
                show("内层结束");
            });
            System.out.println("作用域退出: isBound=" + REQUEST.isBound());
        } finally {
            LOCAL.remove();
            System.out.println("线程局部已remove");
        }
    }

    private static void show(String phase) {
        System.out.println(phase + ": " + REQUEST.get());
    }
}
```

preview/src/main/java/org/example/jvminternals/ch29/StructuredJourney.java

```
package org.example.jvminternals.ch29;

import java.time.Duration;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.StructuredTaskScope;
import java.util.concurrent.TimeUnit;

/** JDK 25第五次预览：任务继承上下文，主动失败与协作取消独立于普通编译。 */
public final class StructuredJourney {
    private static final ScopedValue<String> REQUEST = ScopedValue.newInstance();

    private StructuredJourney() { }

    /** 不带参数为成功路径；fail显式选择已授权的主动异常案例。 */
    public static void main(String[] args) throws InterruptedException {
        boolean fail = args.length == 1 && args[0].equals("fail");
        ScopedValue.where(REQUEST, "request-29").call(() -> {
            runScope(fail);
            return null;
        });
        System.out.println("绑定退出：isBound=" + REQUEST.isBound());
    }

    private static void runScope(boolean fail) throws InterruptedException {
        CountDownLatch started = new CountDownLatch(1);
        try (var scope = StructuredTaskScope.open(
                StructuredTaskScope.Joiner.<String>awaitAllSuccessfulOrThrow(),
                config -> config.withName("request-
scope").withTimeout(Duration.ofSeconds(5)))) {
            var slow = scope.fork(() -> {
                System.out.println("子任务slow上下文=" + REQUEST.get());
                started.countDown();
                try {
                    Thread.sleep(fail ? 3_000L : 100L);
                    return "slow-result";
                } catch (InterruptedException interrupted) {
                    System.out.println("子任务slow收到中断");
                    throw interrupted;
                } finally {
                    System.out.println("子任务slow finally退出");
                }
            });
            var fast = scope.fork(() -> {
                started.await(1, TimeUnit.SECONDS);
                System.out.println("子任务fast上下文=" + REQUEST.get());
```

```
                if (fail) {
                    System.out.println("子任务fast主动抛出教学异常");
                    throw new IllegalStateException("教学故障: fast");
                }
                return "fast-result";
            });
            try {
                scope.join();
                System.out.println("汇合结果: " + slow.get() + "," + fast.get());
            } finally {
                System.out.println("拥有者即将close");
            }
        } catch (StructuredTaskScope.FailedException failed) {
            System.out.println("close完成后接收失败: " + failed.getCause());
        }
        System.out.println("任务作用域已关闭");
    }
}
```